



Course: AI and ML for Robotics

Lab Sheet 7: Deep Reinforcement Learning for Path Planning

Objective:

To implement Deep Reinforcement Learning for Path Planning.

Tools and Libraries Required:

Programming language: Python

Platform : Google Colab T4 GPU

Simulation : PyBullet

Design:

State : 4 dimensional vector : [dx, dy, distanceToGoal, distanceToNearestObstacle]

Action : Continuous value in 2D [-1, +1]

Output : Policy that maps State to Action oriented toward Goal.

Implementation pipeline

1. **Environment setup** — initialize `PyBulletPointEnv`: store start/goal/map, define action/observation spaces, connect to PyBullet and build the scene (plane, robot, obstacles).
2. **Scene markers and debug state** — create start/goal visual markers and initialize trajectory bookkeeping (`path`, `_debug_line_ids`).
3. **Reset** — place robot at start, clear trajectory lines and return initial observation.
4. **State computation** — `_get_state()` computes relative goal vector, distance, and nearest-obstacle distance and returns a 4-D observation.
5. **Action handling** — in `step(action)`: clip action, scale to a max step, compute candidate next position, clip to map bounds, and check collision.
6. **Physics update** — if no collision, update robot position in PyBullet and append the new position to `path`. Draw a persistent debug line segment for the step.
7. **Reward and termination** — compute reward (goal bonus, progress-based reward, step cost, obstacle proximity penalty, collision penalty) and set “done” when goal reached, collision occurs, or max steps exceeded.
8. **Return from step** — return next observation, scalar reward, done flag, False for truncated, and an info dict containing `{"collision": collision}`.
9. **Rendering and recording** — `render()` captures an RGB camera image from a fixed viewpoint; `demo_and_record()` runs the agent, logs steps, collects frames, and writes a video.
10. **Cleanup** — `close()` disconnects the PyBullet client.



Practice Questions:

1. Rewards are hardcoded in above code. Instead dynamic reward can be designed as a heuristic function considering the dynamic obstacles, Goals etc.,
 2. Replace the 4-dim state with a 1D LIDAR array and train the same PPO; Compare sample efficiency and success rate.
 3. Train with moving obstacles to learn dynamic avoidance.
 4. Use the trained policy as a steering function inside RRT: sample nodes, but use the policy to propose local motions between nodes. Measure planning time and path quality.
 5. Compare PPO vs SAC on the same environment and plot learning curves.
-